Grogoni doc 4 June

Grogoniclaw Runtime Specification v1.0

Document Status

Document Title: Grogoniclaw Runtime Specification v1.0

Document Type: AI Runtime Architecture & Execution Specification

Runtime Name: Grogoniclaw

Parent Documents

- Document 1 — Grogoni Architecture v3.0
- Document 2 — Technical Architecture Specification v1.0
- Document 3 — Product Requirements Document & Development Blueprint v1.0

---

1. Purpose

Grogoniclaw is the AI runtime that powers Grogoni.

Unlike an LLM, Grogoniclaw is not a model.

It is an execution runtime responsible for:

- Understanding intent
- Planning work
- Coordinating models
- Managing memory
- Executing tools
- Verifying actions
- Learning business vocabulary
- Maintaining business context
- Routing tasks
- Managing installed skills

Grogoniclaw should be model-agnostic.

It should operate with:

- Small local SLMs
- Medium local models
- Cloud-hosted LLMs
- Future multimodal models

without changing the runtime architecture.

---

2. Runtime Philosophy

Grogoniclaw follows seven principles.

1. Model Agnostic

Models can change.

Runtime remains.

---

2. Tool First

Models never directly modify business data.

Everything happens through tools.

---

3. Deterministic Business Logic

Business rules never depend on model output.

AI proposes.

Business runtime validates.

---

4. Event Driven

Everything produces events.

---

5. Memory Native

Memory is a runtime service.

Not prompt engineering.

---

6. Mobile First

Runtime must work on low-end Android devices.

---

7. Cloud Optional

Every capability should degrade gracefully.

---

3. Runtime Overview

User

↓

Conversation

↓

Intent Router

↓

Planner

↓

Task Scheduler

↓

Model Router

↓

Tool Executor

↓

Verification

↓

Memory

↓

Response Generator

↓

Business Runtime

↓

Sync

---

4. Runtime Components

Grogoniclaw Runtime

├── Conversation Manager

├── Context Builder

├── Intent Router

├── Planner

├── Task Scheduler

├── Model Router

├── Memory Engine

├── Knowledge Engine

├── Skill Manager

├── Tool Executor

├── Verification Engine

├── Event Publisher

├── Response Generator

└── Runtime Monitor

Each component is independent and communicates through runtime events.

---

5. Runtime Lifecycle

Every user request follows a defined lifecycle.

Receive Input

↓

Normalize

↓

Context Build

↓

Intent Detection

↓

Planning

↓

Scheduling

↓

Tool Selection

↓

Execution

↓

Verification

↓

Memory Update

↓

Response

↓

Event Publication

No component may skip verification before a business-changing action.

---

6. Conversation Manager

Responsibilities

- Receive messages
- Detect language
- Detect conversation session
- Track current task
- Handle interruptions
- Recover abandoned tasks

Conversation state example:

```
{
  "session_id": "...",
  "language": "sw-KE",
  "current_goal": "create_invoice",
  "open_entities": [
    "John",
    "Maize"
  ],
  "pending_confirmation": true
}
```

---

7. Context Builder

Purpose

Construct the minimum context required for reasoning.

Inputs

- Recent conversation
- Business memory
- Customer memory
- Current document
- Current invoice
- Installed skills
- Runtime state

Output

Compressed Runtime Context

The Context Builder should summarize rather than replay long histories to reduce token usage.

---

8. Intent Router

The Intent Router classifies requests into executable intents.

Examples:

- CreateInvoice
- AddProduct
- UpdatePrice
- RecordPayment
- CheckDebt
- ImportDocument
- SendReminder
- GenerateReport

Confidence policy:

- ≥0.80: execute planning
- 0.50–0.79: ask clarifying question if ambiguity affects execution
- <0.50: offer guided choices
- Repeated failures: transition to structured form flow

The router must support multilingual and mixed-language input (e.g., English–Swahili).

---

9. Planner

The Planner converts an intent into an execution plan.

Example:

Intent: CreateInvoice

Plan:

1. Resolve customer.
2. Resolve products.
3. Validate inventory.
4. Apply pricing.
5. Calculate taxes.
6. Create invoice.
7. Ask for delivery method.
8. Offer payment link.

The planner never executes actions directly.

---

10. Task Scheduler

The scheduler optimizes execution.

Task categories:

- Sequential
- Parallel
- Deferred
- Background
- Cloud-required

Example:

Resolve Customer
        |
Resolve Products
        |
————————Parallel————————
        |
Calculate Tax
Generate Invoice PDF
        |
Store Invoice

Scheduling considers:

- Device resources
- Battery
- Connectivity
- Latency targets
- User priority

---

11. Model Router

The Model Router selects the most appropriate model.

Candidate models:

- Tiny local model
- Small local model
- Vision model
- Speech model
- Cloud reasoning model

Selection factors:

- Task type
- Required accuracy
- Device RAM
- CPU capability
- Battery level
- Connectivity
- Privacy requirements
- Cost policy

The routing decision is independent of business logic.

---

12. Memory Engine

Memory is a persistent runtime service.

Memory types:

- Conversation Memory
- Business Memory
- Customer Memory
- Knowledge Memory
- Skill Memory

Operations:

- Store
- Retrieve
- Summarize
- Merge
- Expire
- Protect

High-value memories should survive across sessions, while transient conversation state may expire.

---

13. Knowledge Engine

The Knowledge Engine derives insights from structured business data.

Responsibilities:

- Aggregate events
- Generate summaries
- Detect trends
- Produce forecasts
- Recommend actions

Outputs include:

- Top-selling products
- Slow-moving stock
- Customer payment behaviour
- Inventory forecasts
- Profit trends

Knowledge is read-only during inference; updates are generated from validated business events.

---

14. Skill Manager

Skills extend runtime capabilities.

Lifecycle:

Install
↓

Validate
↓

Register

↓

Execute
↓

Update
↓

Disable
↓

Remove

Rules:

- Skills declare permissions.
- Skills call tools rather than databases.
- Skills are isolated from each other.
- Skills are versioned and signed.

---

15. Tool Executor

The Tool Executor is the only component allowed to perform business actions.

Execution pipeline:

Tool Request
↓

Input Validation
↓

Permission Check
↓

Business Rule Check
↓

Execute
↓

Verify
↓

Emit Event
↓

Return Result

Risk levels:

- Low
- Medium
- High
- Critical

High and critical actions require user confirmation.

---

16. Verification Engine

The Verification Engine ensures that proposed actions comply with system rules.

Checks include:

- User permissions
- Role permissions
- Business constraints
- Inventory availability
- Credit limits
- Tax rules
- Fraud indicators
- Confirmation requirements

If any check fails, execution is halted and the user is informed.

---

17. Event Publisher

Every successful business mutation emits an immutable event.

Examples:

- InvoiceCreated
- PaymentReceived
- ProductUpdated
- StockAdjusted
- SkillInstalled
- MemoryUpdated

These events drive synchronization, reporting, notifications, and knowledge generation.

---

18. Response Generator

The Response Generator converts structured execution results into natural language.

Responsibilities:

- Explain completed actions
- Highlight warnings
- Request confirmations
- Suggest next steps

Responses should remain concise while exposing important business details.

---

19. Runtime State Machine

High-level states:

Idle
↓

ReceivingInput
↓

Planning
↓

WaitingForTool
↓

Executing
↓

WaitingForConfirmation
↓

Completed

or

Failed

Each transition is logged for observability.

---

20. Error Recovery

Runtime failures are classified as:

- Intent failure
- Tool failure
- Permission failure
- Validation failure
- Network failure
- Model failure
- Sync failure

Recovery policy:

- Retry transient failures.
- Ask clarifying questions for ambiguous intent.
- Escalate to structured UI when repeated failures occur.
- Never fabricate successful execution.

---

21. Learning Mechanisms

Grogoniclaw learns through explicit business interactions rather than modifying foundation models.

Supported mechanisms:

- Vocabulary aliases
- User corrections
- Preferred suppliers
- Preferred products
- Common workflows
- Frequently used reports

Learning is scoped to the business account.

---

22. Runtime Telemetry

Metrics collected:

- Intent accuracy
- Planning latency
- Tool execution time
- Verification failures
- Model routing decisions
- Local vs cloud inference ratio
- Memory retrieval latency
- Skill usage
- User confirmation frequency

These metrics support optimization without exposing sensitive business data.

---

23. Security Model

Core principles:

- Models never receive raw credentials.
- Tool permissions are least-privilege.
- Sensitive actions require explicit confirmation.
- Memory access is account-scoped.
- Skills are sandboxed.

- Every runtime action is auditable.

---

24. Deployment Modes

Grogoniclaw supports multiple deployment targets.

Edge Mode

- Tiny local model
- Offline-first
- Minimal memory
- Limited tools

Hybrid Mode

- Local planning
- Cloud reasoning
- Background sync

Cloud Mode

- Large models
- Complex workflows
- Heavy document processing
- Marketplace verification

The runtime behaviour remains consistent across all modes.

---

25. Extensibility

Future runtime capabilities include:

- Multi-agent collaboration
- Voice-first execution
- Federated memory synchronization
- Autonomous workflow orchestration
- Predictive planning
- Multi-modal reasoning

- Distributed task execution
- Cross-business collaboration

These features should be added through new runtime modules rather than altering the core execution lifecycle.

---

26. Runtime Governance

Grogoniclaw is governed by the following invariant rules:

1. The runtime is model-agnostic.
2. Business logic remains deterministic.
3. Tools are the only execution mechanism.
4. Verification precedes state-changing actions.
5. Memory is structured and scoped.
6. Events are immutable.
7. Skills are isolated and permissioned.
8. The runtime must operate on low-end devices with graceful degradation.
9. User confirmation is mandatory for high-risk operations.
10. Every action is observable and auditable.

---

27. Final Runtime Vision

Grogoniclaw is not an LLM wrapper.

It is an AI execution runtime that coordinates models, tools, memory, knowledge, skills, and business workflows through deterministic orchestration.

Within Grogoni, Grogoniclaw provides the intelligence layer while the Business Runtime provides trusted business logic. This separation allows the platform to evolve with future AI models without compromising reliability, auditability, or user trust.

Long term, Grogoniclaw should be reusable beyond Grogoni as a general-purpose runtime for AI-native applications that require structured execution, persistent memory, event-driven orchestration, and safe interaction with external systems.